System Architecture

Pipeline Overview

The boon-academy-intervention pipeline processes student engagement data in five sequential stages, transforming raw CSV exports into a prioritised facilitator action list with AI-drafted WhatsApp messages. Each stage is a discrete Python module with a single public function; modules communicate only through the shared DataFrame and a run_log dictionary - no shared mutable state, no database, no inter-process messaging.

- ingestion.py - Merges three source CSVs (student metadata, daily metrics, facilitator notes) into one clean, deduplicated DataFrame. Runs first, before any scoring or AI work.

- risk_engine.py - Applies a deterministic weighted formula to produce a numeric risk_score (0 - 100) and a categorical risk_level (CRITICAL / HIGH / MEDIUM / LOW) for every student. Pure function, no I/O, no randomness.

- llm_engine.py - Calls the Anthropic Claude API once per campus (not once per student) for all CRITICAL and HIGH students on that campus. Writes facilitator_summary and whatsapp_message columns back into the DataFrame.

- output_generator.py - Writes all six output artefacts: intervention_priority_list.xlsx, one campus dashboard per campus, whatsapp_messages.csv, run_log.json, dashboard.html, and intervention_report.docx.

- doc_generator.py - Writes nine documentation files: analysis.md, analysis.docx, and seven technical .docx files. Called after write_outputs() completes; uses the same df and run_log already in scope.

- ASCII pipeline diagram: [student_metadata.csv] [daily_metrics.csv] [facilitator_notes.csv]

```
          \                |                /
           \               |               /
            +---------> ingestion.py <-----------+
                            |
                            v  (clean merged DataFrame)
                       risk_engine.py
                            |
                            v  (+ risk_score, risk_level)
                        llm_engine.py
                            |
                            v  (+ facilitator_summary, whatsapp_message)
                     output_generator.py
                            |
                            v  (6 output files in outputs/)
                        doc_generator.py
                            |
                            v  (9 documentation files)
```

Component Descriptions

Each module encapsulates a single responsibility and exposes exactly one public function. This makes unit testing straightforward: each module can be tested in isolation by supplying a synthetic DataFrame and asserting on the returned DataFrame or file artefact, without spinning up the full pipeline.

- ingestion.ingest(data_paths) -> DataFrame - Reads CSVs with explicit dtypes to prevent ID and phone number coercion, merges on student_id, deduplicates by keeping the first occurrence of each duplicate, fills missing numeric activity columns with zero, and logs data quality warnings for every anomaly found.

- risk_engine.score_risk(df) -> DataFrame - Calculates four component scores (attendance, practice, trend, notes) using configurable weights from config.py, sums them into risk_score, assigns risk_level via threshold comparisons, and appends a recommended_action string. Accepts an optional today parameter for deterministic testing with freezegun.

- llm_engine.enrich_with_llm(df, api_key, http_client=None) -> tuple[DataFrame, dict] - Groups CRITICAL/HIGH students by campus_id, constructs a cohort prompt per campus, calls the Claude API with tool-use forcing, parses the structured response, and writes results back. Never raises an exception; uses a three-layer fallback. Returns a counts dict (api_calls_made, tokens_used, fallbacks_triggered).

- output_generator.write_outputs(df, output_dir, run_log) -> dict[str, Path] - Creates the output directory, then calls six private helpers in order. Returns a dict mapping named keys to their file paths for main.py logging.

- doc_generator.write_docs(df, run_log, docs_dir) -> dict[str, Path] - Creates the docs directory, loads docs_content.yaml once, then calls nine private helpers. Returns a dict mapping named keys to file paths.

Technology Choices

Technology choices were made to minimise operational complexity for a team running daily batch jobs on a single facilitator machine, while keeping the dependency footprint auditable and the test suite reliable. Every version is pinned in requirements.txt.

- pandas 2.2.3 (not 3.x) - Copy-on-Write is opt-in in 2.x and mandatory in 3.x. Upgrading to 3.x would break chained assignment patterns used in ingestion and risk_engine without any functional benefit at current scale.

- openpyxl 3.1.5 (not xlsxwriter) - xlsxwriter is write-only; openpyxl supports read, write, and append. Test assertions open the generated .xlsx files and inspect cell values directly, which requires read support.

- python-docx 1.1.2 (not 1.2.0) - Version 1.2.0 has an open issue with OxmlElement and table border rendering. Pinned at 1.1.2 until upstream fixes are released and verified.

- PyYAML 6.0.3 - Used for loading llm_templates.yaml and docs_content.yaml via yaml.safe_load(). The safe_load() call prevents arbitrary code execution from a tampered YAML file; yaml.load() is never used.

- anthropic 0.103.1 - The official Anthropic Python SDK. Tool-use (structured JSON output) is used instead of text parsing, eliminating prompt-injection and markdown-stripping edge cases.

- respx 0.23.1 - HTTP mocking library for tests. The Anthropic SDK uses httpx internally; the responses library intercepts only requests-based calls and silently misses SDK calls. respx mocks at the httpx transport layer, which is exactly where the SDK sends requests.

- Jinja2 3.1.6 - Used only for dashboard.html.j2. Loaded lazily inside _write_html_dashboard() rather than at module import so the dependency is not paid unless the HTML output is requested.

Why Not n8n/Zapier or Airtable

Several no-code and low-code platforms were evaluated against the specific requirements of the intervention pipeline. All were rejected on the same fundamental grounds: they trade debuggability, version-control, and offline reliability for ease of initial setup - a tradeoff that is wrong for a daily batch job that must be auditable by a part-time technical team at 20 campuses.

- n8n / Zapier - Workflow logic is stored as visual node graphs, not text. The pipeline cannot be code-reviewed, diffed in git, or unit-tested. Debugging a failed run requires navigating a web UI rather than reading a log file. Breaks under API rate limits with no programmatic fallback mechanism.

- Airtable - Paid tier required for API access and automation. Vendor lock-in means student PII is stored on a third-party server, raising data retention concerns. No offline operation - if the Airtable service is unavailable, the entire workflow stops.

- Google Sheets + Apps Script - Viable for very small scale (< 50 students) but becomes slow and unreliable at 300+ rows with formula dependencies. No meaningful test infrastructure; debugging requires manual inspection.

- The Python batch approach chosen here: fully version-controlled, runs offline, has 111 automated tests, produces structured logs, and costs nothing beyond the Anthropic API calls ($200/month budget covers the projected load at 100 campuses).